

Selenium + TestNG Interview Questions

Oct 5, 2026

These are 104 questions asked in Selenium + TestNG interviews, each with a short model answer. Where it helps, the answer points to how your own project, [selenium-java-framework](#), does it. A concrete example from your own code is the strongest answer you can give.

Level	Sections	What interviewers check
Basic	WebDriver basics, Locators, TestNG core	You know the API and the vocabulary
Intermediate	Waits, Interactions, TestNG data and execution, Maven	You can write stable tests and run them in many ways
Advanced	Framework design, Troubleshooting, Your project	You can design a framework, explain trade-offs and debug failures

Versions used in the answers: Selenium 4, TestNG 7, Java 17.

1. Selenium WebDriver basics

Q1. What is Selenium, and what are its components? Selenium is an open source tool for automating browsers. Its parts are WebDriver (the API that drives a browser), Grid (runs tests on remote machines in parallel) and IDE (a record-and-playback browser extension).

Q2. How does Selenium 4 WebDriver talk to the browser? Your code calls the language bindings. They send W3C WebDriver protocol commands over HTTP to the browser's driver, such as chromedriver, and the driver controls the browser. Selenium 4 uses the W3C protocol only; the old JSON Wire Protocol is gone.

Q3. What changed in Selenium 4? It follows the W3C standard, adds relative locators (`above` , `near` , `toLeftOf`), lets you open a new tab or window with `newWindow` , can screenshot a single element, and gives access to Chrome DevTools (CDP) and the newer BiDi protocol. Selenium Manager also downloads drivers automatically.

Q4. What is Selenium Manager? It is built into Selenium 4.6 and later. It finds or downloads the driver that matches the installed browser, so you no longer need WebDriverManager or a driver `.exe` in the project. *In my project*: no driver binaries are stored; `.gitignore` even excludes `*.exe` .

Q5. What is the difference between `WebDriver` and `WebElement`? `WebDriver` controls the browser: navigation, windows, finding elements. `WebElement` is one element on the page: click, type, read text and attributes.

Q6. What is the difference between `driver.close()` and `driver.quit()`? `close()` closes the current window only. `quit()` closes every window and ends the driver session. Always use `quit()` in teardown, or driver processes are left running.

Q7. What is the difference between `findElement` and `findElements`? `findElement` returns the first match and throws `NoSuchElementException` if nothing matches. `findElements` returns a list, which is empty when nothing matches. Use `findElements` to check that something is absent without catching an exception.

Q8. What is the difference between `get()` and `navigate().to()`? Both load a URL and wait for the page load. `navigate()` also offers `back()`, `forward()` and `refresh()` for the browser history.

Q9. How do you run tests headless, and why? Add `--headless=new` to `ChromeOptions` (or `-headless` for Firefox). It is faster and works on CI machines with no display. Set the window size explicitly, because the default headless window is small. *In my project:* `-Dheadless=true` switches it on, and `DriverManager` sets 1920×1080 for every browser.

Q10. What are browser options (capabilities) used for? They configure the browser before it starts: headless mode, window size, language, downloads and disabled pop-ups. *In my project:* Chrome gets `--disable-features=PasswordLeakDetection`, because Chrome's "change your password" warning for the demo password covered the page.

Q11. What can't Selenium test? CAPTCHAs, desktop apps, native mobile apps (use Appium), images and visual layout (use a visual testing tool), performance under load (use JMeter or Gatling), and OS dialogs such as the Windows file picker.

2. Locators

Q12. Which locator strategies does Selenium support? `id`, `name`, `className`, `tagName`, `linkText`, `partialLinkText`, `cssSelector` and `xpath`, plus relative locators in Selenium 4.

Q13. Which locator do you prefer, and why? A stable `id`, `name` or test attribute such as `data-testid` first, then CSS, then XPath. Choose what is least likely to change, not what is quickest to copy from DevTools. *In my project:* the login fields use `By.name("username")`.

Q14. CSS selector or XPath: what is the difference? CSS is shorter and often faster, but it can only move down the DOM and cannot match on visible text. XPath can move up to parents (`ancestor::`), across to siblings and match text (`normalize-space()='Save'`).

Q15. What is the difference between absolute and relative XPath? Absolute XPath starts at the root (`/html/body/div[2]/...`) and breaks on any layout change. Relative XPath starts with `//` and targets attributes or text, so it survives layout changes. Never use absolute XPath in a framework.

Q16. Name some useful XPath functions and axes. `contains()`, `starts-with()`, `normalize-space()`, `text()`, and the axes `ancestor`, `parent`, `following-sibling`, `preceding-sibling` and `descendant`.

Q17. How do you handle elements whose id changes on every load? Locate by something stable: a fixed part of the attribute (`contains(@id, 'user')`), the visible text, a nearby label, or a `data-*` attribute. Ask developers to add test ids where nothing is stable.

Q18. How do you find an input when the label is not linked to it? Start at the label text and move up to the container that holds both, then down to the input. *In my project:* `BasePage.inField(label, inner)` builds `//label[normalize-space()='Username']/ancestor::div[contains(@class, 'oxd-input-group')]/input`, because OrangeHRM does not link labels to inputs.

Q19. How do you find a row in a table by its content? Use XPath on the row that contains the cell text, then go down to the button you need. *In my project:* `EmployeeListPage.delete(id)` clicks the trash button of the row that has a cell with exactly that employee ID. The table is also read as a list of maps, column title → text, so tests never use cell positions.

Q20. What are relative locators? Selenium 4 locators based on screen position: `RelativeLocator.with(By.tagName("input")).below(label)`. They are useful when the DOM gives no good hook, but they depend on layout, so use them sparingly.

3. Waits and synchronization

Wait	Set on	Waits for	Use it when
Implicit	The driver, once	Any element to be present, on every <code>findElement</code>	Rarely; it hides timing and mixes badly with explicit waits
Explicit (<code>WebDriverWait</code>)	One action	A condition: visible, clickable, URL, text	Default choice for every action
Fluent (<code>FluentWait</code>)	One action	A condition, with your own polling interval and ignored exceptions	Custom polling or exceptions to ignore

Q21. Why should you avoid `Thread.sleep()` ? It always waits the full time: too long when the page is fast, too short when it is slow. Tests become both slow and flaky. *In my project:* there is no `Thread.sleep` and no implicit wait; every action in `BasePage` uses an explicit wait.

Q22. Why not mix implicit and explicit waits? The implicit wait runs inside each poll of the explicit wait, so timeouts add up in ways that are hard to predict. Selenium's own docs warn against mixing them.

Q23. Name some common `ExpectedConditions`. `visibilityOfElementLocated`, `elementToBeClickable`, `presenceOfElementLocated`, `invisibilityOfElementLocated`, `urlContains`, `textToBePresentInElement`, `alertIsPresent` and `frameToBeAvailableAndSwitchToIt`.

Q24. What is the difference between presence and visibility? Presence means the element is in the DOM. Visibility means it is also displayed with a size greater than zero. You can only click or type into a visible element.

Q25. What is a `StaleElementReferenceException`, and how do you fix it? The element you hold was removed or re-rendered, so the reference points at nothing. Find it again just before you use it, and let waits retry. *In my project:* OrangeHRM re-renders tables after each search, so `BasePage` calls `wait.ignoring(StaleElementReferenceException.class)`, and `tableRows()` reads the whole table inside one wait, so a re-render in the middle simply retries.

Q26. How do you wait for a loading spinner? Wait for it to become invisible. The catch: if you check too early, the spinner has not appeared yet and the wait passes at once. *In my project:* `waitForLoader()` first gives the spinner up to 2 seconds to appear, then waits for it to disappear.

Q27. How do you write a custom wait condition? Pass a lambda to `wait.until`. Return `null` or `false` to keep polling, or a value to finish. *In my project:* `EmployeeProfilePage.fullName()` waits until the name header has text, because the header is drawn before the name arrives.

Q28. What exception do you get when an explicit wait times out? `TimeoutException`. Its message names the condition and the locator, which is the first clue when you debug.

4. WebDriver interactions

Q29. How do you handle a `<select>` dropdown? Wrap it in `new Select(element)`, then call `selectByVisibleText`, `selectByValue` or `selectByIndex`. `getFirstSelectedOption()` reads the current choice.

Q30. How do you handle a custom dropdown that is not a `<select>`? `Select` does not work on it. Click the dropdown to open it, then click the option by its text. *In my project:* OrangeHRM's dropdowns are `div`s, so `BasePage.choose("User Role", "Admin")` clicks the field and then the `role='option'` with that text.

Q31. How do you handle JavaScript alerts? `driver.switchTo().alert()`, then `accept()`, `dismiss()`, `getText()` or `sendKeys()` for a prompt. Wait with `ExpectedConditions.alertIsPresent()` first.

Q32. How do you work with iframes? Switch in with `driver.switchTo().frame(...)` (by index, name/id or element), work inside it, then return with `defaultContent()` or `parentFrame()`. Elements inside a frame cannot be found until you switch.

Q33. How do you switch between windows or tabs? Read `getWindowHandle()` before the click and `getWindowHandles()` after it, then switch to the new handle. In Selenium 4, `driver.switchTo().newWindow(WindowType.TAB)` opens and switches in one step.

Q34. When do you use the `Actions` class? For mouse and keyboard gestures: hover (`moveToElement`), right-click (`contextClick`), double-click, drag and drop, and key chords. End the chain with `perform()`.

Q35. When do you use `JavascriptExecutor`, and what is the risk? To scroll an element into view, read values the API does not expose, or click when an overlay blocks a normal click. The risk is that a JavaScript click skips what a real user would hit, so it can hide real bugs. Use it as a last resort.

Q36. `clear()` does not empty a field. Why, and what do you do? In React and Vue apps the page's form model may not see the change that `clear()` makes, and the old value comes back. Select all and delete instead. *In my project:* `BasePage.type()` sends Ctrl+A and Delete before typing, because OrangeHRM's Vue form kept the old value.

Q37. How do you take a screenshot? Cast the driver to `TakesScreenshot` and call `getScreenshotAs(OutputType.FILE)`, `BYTES` or `BASE64`. Selenium 4 can also capture one element. *In my project:* `BaseTest` captures Base64 when a test fails, saves a PNG to `target/screenshots`, and embeds it in the HTML report.

Q38. How do you upload a file? Call `sendKeys("C:\\path\\file.pdf")` on the `<input type='file'>`. Selenium cannot drive the OS file picker. On Grid, set a `LocalFileDetector` so the file is sent to the remote machine.

Q39. How do you read text, attributes and properties? `getText()` returns visible text. `getAttribute()` is the older catch-all. Selenium 4 adds `getDomAttribute()` for the HTML attribute and `getDomProperty()` for the live value. *In my project:* `valueOf()` reads an input with `getDomProperty("value")`.

5. Page Object Model and framework design

Q40. What is the Page Object Model, and why use it? Each page gets a class that holds its locators and actions; tests call those methods and only assert. When the UI changes, you fix one page class, not every test. Tests also read like the scenario.

Q41. Should page objects contain assertions? Generally no. Pages return data or the next page, and tests decide what is right. *In my project:* pages return values such as `errorMessage()` or `usernames()`, and only the test classes call `assertEquals`.

Q42. What is Page Factory, and do you use it? Page Factory creates elements with `@FindBy` and `PageFactory.initElements`. Many teams avoid it now: its lazy proxies go stale easily and hide when a lookup happens. *In my project:* locators are `private static final By` fields, found inside explicit waits each time.

Q43. What is method chaining (a fluent page API)? A method returns the page you land on, so steps chain:

`loginAsAdmin().openAdmin().filterByUsername("Admin").search()`. Navigation is explicit, and the compiler stops you from calling a method on the wrong page.

Q44. What goes into a BasePage? The wait helpers and the widgets every page shares. *In my project:* `BasePage` holds `click`, `type`, `waitVisible`, `waitForLoader`, labelled fields, dropdowns, tables and toasts. `AppPage` extends it with the side menu and user menu for every page after login.

Q45. How do you make the driver safe for parallel runs? Keep one driver per thread in a `ThreadLocal<WebDriver>`, never in a static field. *In my project:* `DriverManager` holds a `ThreadLocal`, quits any browser the thread still had open, and calls `remove()` after `quit()`.

Q46. How do you manage configuration such as URL, browser and timeouts? Put defaults in a properties file and let system properties override them, so CI can switch values without code changes. *In my project:* `Config` reads `config.properties` from the classpath, and any key can be overridden with `-Dkey=value`, such as `-Dbrowser=edge`.

Q47. Where do you keep test data? Outside the test logic: data providers, properties, JSON or Excel files, or generated values. Generate unique data for anything you create, so parallel runs and reruns never collide. *In my project:* `TestData.newEmployee()` builds a random ID, `QA` plus 8 digits, and the test deletes the employee again.

Q48. How do you keep tests independent? Each test gets its own browser and data, logs in itself, and does not rely on another test's result or order. *In my project:* every method gets a fresh browser in `@BeforeMethod`, and checks never count shared records on the public demo site.

Q49. What design patterns are used in Selenium frameworks? Page Object, Factory (create a driver by browser name), Singleton (one config), Builder or fluent interfaces (chained pages), and Strategy (swap browsers or environments).

Q50. How do you log test steps? Record each user action in the report, not only the failures. *In my project:* page methods call `Steps.log("Filter by username \"Admin\")`, which goes through TestNG's `Reporter.log`, and the report lists every step in order with passwords masked.

6. TestNG core

Q51. What is TestNG, and why use it over JUnit 4? TestNG is a Java test framework with groups, dependencies, data providers, parallel runs, listeners and XML suites built in. JUnit 5 has caught up on many of these, but TestNG's groups, `dependsOnMethods` and suite files remain common in UI automation.

Q52. In what order do TestNG annotations run?

1. `@BeforeSuite`
2. `@BeforeTest` (once per `<test>` tag in `testng.xml`)
3. `@BeforeClass`
4. `@BeforeMethod`
5. `@Test`
6. `@AfterMethod`
7. `@AfterClass`
8. `@AfterTest`
9. `@AfterSuite`

`@BeforeGroups` and `@AfterGroups` run around the first and last test of a group.

Q53. What is the difference between `@BeforeTest` and `@BeforeMethod`? `@BeforeTest` runs once per `<test>` block in `testng.xml`. `@BeforeMethod` runs before every `@Test` method. *In my project:* `BaseTest` starts a browser in `@BeforeMethod`, so every test gets a fresh one.

Q54. What does `alwaysRun = true` do on a configuration method? The method runs even when you filter by groups, and an `@After` method still runs after a failure. *In my project:* `startBrowser` and `stopBrowser` use it, so a browser is always started and always closed.

Q55. What are groups, and how do you run them? Labels on tests, such as `smoke` or `regression`. Run them with `<groups><run><include name="smoke"/></run></groups>` in `testng.xml`, or `-Dgroups=smoke` with Maven. *In my project:* the groups are `smoke`,

`login`, `admin`, `pim` and `demo`; `demo` is excluded by default.

Q56. What do `priority`, `enabled`, `timeOut` and `invocationCount` do? `priority` orders tests in a class (lower runs first; the default is 0). `enabled = false` turns a test off. `timeOut` fails a test that runs longer than the limit in milliseconds. `invocationCount` runs the same test several times.

Q57. Should tests rely on `priority` to run in order? No. Ordered tests depend on each other, so one failure breaks the rest and you cannot run a test alone. Keep each test independent, and use `dependsOnMethods` only when a real dependency exists.

Q58. What does `dependsOnMethods` do when the first test fails? The dependent test is skipped, not failed, and the report says which method it depended on. *In my project:* `ReportDemoTest.skippedBecauseDependencyFailed` shows this on purpose. `alwaysRun = true` would run it anyway.

Q59. What is the difference between hard and soft assertions? `Assert` stops the test at the first failure. `SoftAssert` collects every failure and reports them all when you call `assertAll()`; forget `assertAll()` and the test passes silently. Use soft asserts to check many independent fields in one go.

Q60. How do you skip a test at runtime? Throw `SkipException("reason")` when a precondition is missing, such as a feature that is turned off or a browser that is not supported. *In my project:* `ReportDemoTest.skippedWhenPreconditionIsMissing` does this, and the report shows the reason.

Q61. How do you test that an exception is thrown? Use `@Test(expectedExceptions = IllegalStateException.class)`, optionally with `expectedExceptionsMessageRegExp`, or `Assert.assertThrows(...)` inside the test.

7. TestNG data, suites, parallel runs and listeners

Q62. What is a `@DataProvider`? A method that returns `Object[][]` (or an `Iterator<Object[]>`); the test runs once per row, and each row's values become the test's parameters. *In my project:* `TestData.invalidCredentials` has 3 rows, so `loginIsRejectedForInvalidCredentials` runs 3 times, and the report shows each case by name.

Q63. How do you share a data provider between classes? Put it in its own class as a `static` method and reference it with `dataProviderClass`. *In my project:* `@Test(dataProvider = "invalidCredentials", dataProviderClass = TestData.class)`.

Q64. What is the difference between `@DataProvider` and `@Parameters` ? `@Parameters` reads single values from `testng.xml`, such as a browser or URL for the whole suite. `@DataProvider` feeds many rows of data to one test. `@DataProvider(parallel = true)` also runs the rows in parallel.

Q65. What goes into `testng.xml`? The suite, its `<test>` blocks, classes or packages, groups to include or exclude, parameters, listeners and parallel settings. *In my project:* `testng.xml` runs the tests package with `parallel="classes"` `thread-count="3"` and excludes the `demo` group; it is used for IDE runs.

Q66. What can run in parallel in TestNG? `parallel` can be `methods`, `classes`, `tests` or `instances`, with `thread-count` as the limit. With Selenium, every thread needs its own driver (`ThreadLocal`). *In my project:* classes run in parallel on 3 threads, and `-Dthreads=1` runs them one after another.

Q67. Why `parallel=classes` rather than `methods` ? Methods of the same class then never run at once, which keeps `dependsOnMethods` and class-level state simple, while you still get one browser per thread.

Q68. What TestNG listeners do you know? `ITestListener` (test start, pass, fail, skip), `ISuiteListener`, `IInvokedMethodListener`, `IReporter` (builds a report after the run), `IRetryAnalyzer` and `IAnnotationTransformer`.

Q69. How do you register a listener? With `@Listeners` on a class, `<listeners>` in `testng.xml`, `-listener` on the command line, or Java's ServiceLoader file `META-INF/services/org.testng.ITestNGListener`. *In my project:* the ServiceLoader file is used, so the report works from Maven, `testng.xml` and the IDE without registering it twice.

Q70. How did you build a custom HTML report? *In my project:* `HtmlReportListener` implements `IReporter`. After the run it collects passed, failed and skipped results, the steps from `Reporter.getOutput(result)` and the failure screenshot stored as a result attribute. It writes one self-contained HTML file with a pass-rate ring, search, status filters and a dark mode.

Q71. How do you retry a failed test? Implement `IRetryAnalyzer`. Its `retry(result)` returns `true` while attempts remain. Attach it per test with `retryAnalyzer`, or to all tests with an `IAnnotationTransformer`. Retries hide flaky tests, so log every retry and fix the cause.

Q72. Where does `testng-failed.xml` come from? TestNG writes it after a run that had failures. It lists only the failed tests, so you can rerun just those.

Q73. Why would a test show as skipped? A `SkipException`, a failed method it depends on, a failure in a `@Before` configuration method (everything after it is skipped), or a retry that was replaced by a later attempt.

8. Maven, CI and reporting

Q74. How does Maven run TestNG tests? The Surefire plugin runs them in the `test` phase. It finds `*Test` classes, or the suite files you give it in `suiteXmlFiles`. Maven sees `testng` as a dependency and picks TestNG automatically.

Q75. Why might `-Dgroups=smoke` do nothing? When Surefire runs a suite XML file, the suite decides what runs, and `groups` / `excludedGroups` from the command line are ignored. *In my project:* Maven runs the test classes directly with `parallel=classes`, so `-Dgroups` and `-Dtest` work, and `testng.xml` is kept only for the IDE.

Q76. How do you run one class, one method or one group? `mvn test -Dtest=LoginTest`, `mvn test -Dtest=LoginTest#adminCanLogIn` and `mvn test -Dgroups=smoke`.

Q77. How do you pass settings to tests from the command line? With `-Dkey=value`; Surefire forwards it as a system property, which the framework reads. *In my project:* `-Dbrowser=edge -Dheadless=true -Dthreads=1`.

Q78. What is a Maven profile, and when do you use one? A named set of properties or plugin settings you switch on with `-P`. *In my project:* the `demo` profile sets `excludedGroups` to `none`, so `mvn clean test -Pdemo` also runs the tests that fail and skip on purpose.

Q79. How do you run Selenium tests in CI? Run headless on a machine or container with the browser installed, with `mvn clean test`. Save the reports and screenshots as build artifacts, and publish the JUnit XML that Surefire writes so the CI tool shows each test.

Q80. What is Selenium Grid? A hub that sends each test to a node with the right browser and OS, so tests run on many machines at once. Tests use `RemoteWebDriver` with the hub's URL and browser options. Docker images such as `selenium/standalone-chrome` make it easy to set up.

Q81. Which reports have you used? TestNG's built-in reports, Surefire XML, ExtentReports, Allure, and custom listeners. *In my project:* a custom `Reporter` writes `selenium-test-report.html` with steps, failure messages, filtered stack traces and embedded screenshots.

Q82. What should a good UI test report show for a failure? Which test failed, the steps it ran, the error message with the expected and actual values, a short stack trace, a screenshot of the moment it failed, and the environment: browser, URL and build.

9. Scenario and troubleshooting questions

Q83. A test passes locally but fails in CI. How do you investigate?

1. Read the failure message and screenshot: did the page not load, was an element missing, or was a value wrong?
2. Compare the environments: headless mode, window size, browser version, network speed, and the test data.
3. Run it locally the way CI does (`-Dheadless=true`) and look for timing problems, such as a missing wait.

Q84. How do you deal with flaky tests? Find the cause before you add a retry. Common causes are missing waits, stale elements, shared or changing test data, order dependence and an unstable environment. *In my project:* a search test failed now and then with `StaleElementReferenceException` until the table was read inside the wait.

Q85. Several tests fail at once with timeouts. What do you check first? Whether the application is up, before you suspect the tests. *In my project:* one run had 3 real tests time out, and the failure screenshots showed an nginx **504 Gateway Time-out** from the OrangeHRM demo site. The site recovered a minute later, and the rerun was green.

Q86. A click fails with `ElementClickInterceptedException` . Why? Something covers the element: a spinner, a toast, a cookie banner or a sticky header. Wait for the overlay to disappear, or scroll the element into view. Use a JavaScript click only as a last resort.

Q87. Tests pass alone but fail in parallel. What is wrong? Shared state: a static driver, shared page objects, or several tests changing the same record. Use a `ThreadLocal` driver, new page objects per test, and unique data per test.

Q88. How do you test a flow that creates data on a shared environment? Create your own data with unique values, check it, then delete it in the same test or in cleanup. *In my project:* `EmployeeTest` adds an employee with a random ID, finds it, deletes it and checks that it is gone. If the test fails midway, the employee stays; an `@AfterMethod` cleanup would fix that.

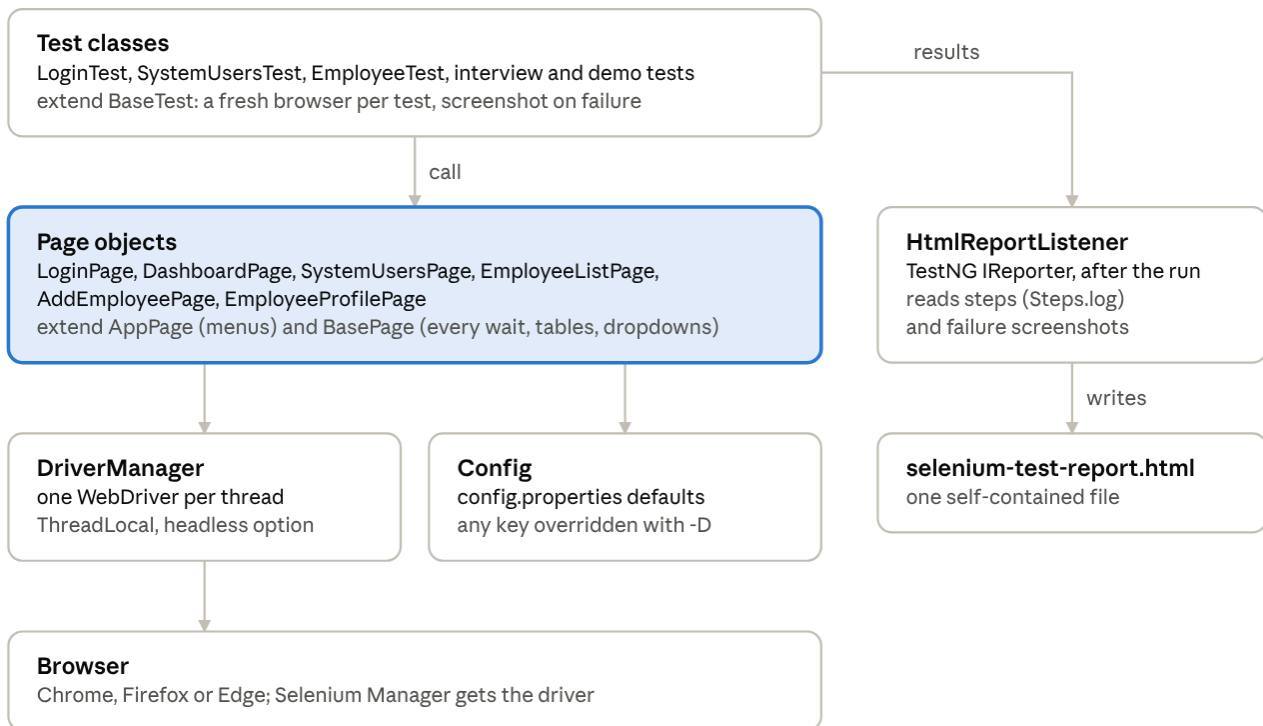
10. Questions about your own project

Q89. Walk me through your framework. A two-minute answer, top down:

1. **Stack:** Java 17, Selenium 4.50, TestNG 7.12 and Maven, testing the OrangeHRM demo site.
2. **Layers:** test classes extend `BaseTest` ; they call page objects; the pages extend `AppPage` and `BasePage` , which hold every wait; `DriverManager` and `Config` sit underneath.
3. **Running it:** test classes run in parallel on 3 threads, each test in its own browser. Groups, browser, headless mode and thread count are switched from the command line.

4. **Reporting:** a custom `Reporter` writes one HTML file with steps, failures and screenshots.
5. **Coverage:** 14 tests for login, admin user search and the PIM employee life cycle; 15 interview-scenario tests for Selenium techniques and TestNG features; plus 4 demo tests that fail and skip on purpose to show the report.

Tests call pages, pages hold every wait, one listener builds the report



selenium-java-framework layers · 7 parts

Tests never touch WebDriver directly. After the run, the report listener reads each test's steps and failure screenshot.

Q90. What would you improve next? A cleanup hook for employees left behind when a test fails midway; the RetryOnce analyzer on every test (through an `IAnnotationTransformer`) for outages of the demo site, with every retry logged; a CI pipeline that publishes the report; and runs on Edge and Firefox through Selenium Grid.

Other questions to prepare from your own code:

- Why did you write your own report instead of using Extent or Allure?
- Why are the demo tests excluded by default, and how do you include them?
- Why does `type()` use Ctrl+A and Delete instead of `clear()` ?
- How does the report get the screenshot from `BaseTest` ? (A result attribute, `HtmlReportListener.SCREENSHOT_ATTRIBUTE`.)
- Why does the login step mask the password in the report?

- Why does `loginAs` stop with the login error instead of waiting 15 seconds for the dashboard?

11. Coding scenarios: Selenium

These are the "write the code" questions. Each answer is a real, passing test in `SeleniumScenariosTest`, with the WebDriver calls in the page objects. The snippets below show the core lines.

Q91. Get all items of a menu (or any list of elements) and check them. `findElements` returns every match; map them to their text. Check with `containsAll` rather than `equals`, so a new menu entry on the site does not break the test.

```
// AppPage: wait until every entry is visible, then read the names
List<String> items =
wait.until(ExpectedConditions.visibilityOfAllElementsLocatedBy(MENU_ITEMS))
    .stream().map(WebElement::getText).map(String::trim).toList();

// Test: the core modules are there, and no entry is blank
assertTrue(items.containsAll(List.of("Admin", "PIM", "Leave", "Time",
"Dashboard")));
assertTrue(items.stream().noneMatch(String::isBlank));
```

On the demo site the menu has 12 entries, from Admin to Buzz.

Q92. Find the broken links on a page. Selenium can read a link's URL but not its HTTP status, so request each URL with Java's `HttpClient` and treat 400 or above as broken. Read the `href` property, which is always absolute; the attribute can be relative. Collect every broken link before you assert, so one failure lists them all.

```
Map<String, Integer> broken = new LinkedHashMap<>();
for (String link : dashboard.menuLinks()) {           // href property of each
menu link
    int status = Links.statusCode(link);               // HttpClient GET,
follows redirects
    if (status >= 400) {
        broken.put(link, status);
    }
}
assertTrue(broken.isEmpty(), "Broken links: " + broken);
```

The HTTP client has no login cookie, so OrangeHRM redirects it to the login page; a 200 after the redirect still proves the link leads somewhere. All 12 menu links return 200.

Q93. Read the options of a drop-down that is not a `<select>` . `Select` only works on `<select>` elements. For a custom drop-down: open it, read the option elements, then close it so the page is left as it was.

```
By field = inField("User Role", "///div[contains(@class,'oxd-select-text')]");
click(field);                                     // open the list
List<String> options = wait.until(ExpectedConditions
    .visibilityOfAllElementsLocatedBy(By.cssSelector("div[role='listbox']
div[role='option']")))
    .stream().map(WebElement::getText).map(String::trim).toList();
click(field);                                     // close it again
assertEquals(options, List.of("-- Select --", "Admin", "ESS"));
```

Q94. Log in using only the keyboard. The `Actions` class builds a chain of input events; nothing is sent until `perform()` .

```
new Actions(driver)
    .click(usernameField)           // focus the first field
    .sendKeys("Admin")
    .sendKeys(Keys.TAB)             // move to the password field
    .sendKeys("admin123")
    .sendKeys(Keys.ENTER)          // submit the form
    .perform();
```

Q95. Open a new tab, work in it, close it and return. Store the first window handle before you open anything. After `close()` the driver points at no window until you switch back.

```
String firstTab = driver.getWindowHandle();
driver.switchTo().newWindow(WindowType.TAB);           // Selenium 4: opens
and switches
new DashboardPage(driver).open();                     // already logged in:
tabs share cookies
assertEquals(driver.getWindowHandles().size(), 2);

driver.close();                                       // closes the current
(second) tab
driver.switchTo().window(firstTab);                   // must switch back
explicitly
```

The answer interviewers look for: tabs in one browser share the session cookie, so the new tab is logged in.

Q96. Use JavaScript to read what WebDriver cannot. Cast the driver to `JavascriptExecutor` ; `return` hands a value back as a Java type.

```
String readyState = (String) ((JavascriptExecutor)
driver).executeScript("return document.readyState;");
assertEquals(readyState, "complete");
```

The framework wraps this as `BasePage.runScript(script, args...)`. Its Javadoc warns that a JavaScript click skips the checks a real click makes.

Q97. Take a screenshot of a single element. Selenium 4 lets any `WebElement` capture itself. Check the result is a real image: every PNG starts with the bytes `89 50 4E 47`.

```
byte[] png =
driver.findElement(By.tagName("form")).getScreenshotAs(OutputType.BYTES);
assertEquals(Arrays.copyOf(png, 4), new byte[] { (byte) 0x89, 'P', 'N', 'G'
});
Files.write(Path.of("target", "screenshots", "login-form.png"), png);
```

12. Coding scenarios: TestNG

Each TestNG feature below is used on a real OrangeHRM check in `TestNgFeaturesTest`. Data providers and groups are in `LoginTest`; a skip and a failed dependency are in `ReportDemoTest`.

Q98. Check several things on a page and report every mismatch at once. Use `SoftAssert`. Each check records its failure and carries on; `assertAll()` at the end fails the test with every mismatch listed. Forget `assertAll()` and the test passes even when checks failed.

```
SoftAssert soft = new SoftAssert(); // a new one per test:
it keeps state
soft.assertEquals(driver.getTitle(), "OrangeHRM", "Page title");
soft.assertEquals(login.placeholder("Username"), "Username", "Username
placeholder");
soft.assertEquals(login.placeholder("Password"), "Password", "Password
placeholder");
soft.assertEquals(login.buttonText(), "Login", "Button text");
soft.assertAll(); // fails here, listing
every mismatch
```

Q99. Write a test that passes only when a specific error happens. `expectedExceptions` names the exception type; `expectedExceptionsMessageRegExp` also checks its message. The test fails if nothing is thrown or the message differs.

```
@Test(expectedExceptions = IllegalStateException.class,
      expectedExceptionsMessageRegExp = ".*Invalid credentials.*")
public void wrongPasswordThrowsTheLoginError() {
    openLoginPage().loginAs("Admin", "wrong-password"); // throws when the
    error banner shows
}
```

Q100. Run one test only after another one passed. Combine `priority` (the order within a class; lower runs first) with `dependsOnMethods` (the actual condition). If the first test fails, TestNG skips the second rather than failing it.

```
@Test(priority = 1)
public void loginPageIsShown() { ... }

@Test(priority = 2, dependsOnMethods = "loginPageIsShown")
public void adminLogsInAfterThePageCheck() { ... }
```

Q101. Run the same test several times. `invocationCount = 2` runs it twice. Each run gets its own `@BeforeMethod` and therefore its own browser, and the report lists each run. Use it to expose a check that only sometimes passes.

```
@Test(invocationCount = 2)
public void wrongPasswordIsRejectedEveryTime() {
    LoginPage login = openLoginPage().submit("Admin", "wrong-password");
    assertEquals(login.errorMessage(), "Invalid credentials");
}
```

Q102. Retry a test that failed because of the environment. Implement `IRetryAnalyzer`. TestNG calls `retry()` after each failed attempt; `true` runs the test again. TestNG keeps one analyzer per test method, so a field can count attempts. The failed attempt shows as skipped in the report, so retries stay visible.

```

public class RetryOnce implements IRetryAnalyzer {
    private static final int MAX_RETRIES = 1;
    private int retries;                                // attempts used for
this test method

    @Override
    public boolean retry(ITestResult result) {
        if (retries < MAX_RETRIES) {
            retries++;
            return true;                                // run the test again
        }
        return false;                                  // report the failure
    }
}

@Test(retryAnalyzer = RetryOnce.class)
public void adminLoginIsRetriedOnceOnFailure() { ... }

```

A retry hides a real bug just as well as a site outage, so log every retry and fix the cause of repeated ones.

Q103. Pass a value from testng.xml into a test. `@Parameters` reads `<parameter>` tags from the suite file. When the test runs without that file (Maven running the classes directly, or the IDE running one class), `@Optional` supplies the default instead of failing.

```
<parameter name="expectedTitle" value="OrangeHRM" />
```

```

@Parameters("expectedTitle")
@Test
public void titleMatchesTheSuiteParameter(@Optional("OrangeHRM") String
expectedTitle) {
    openLoginPage();
    assertEquals(driver.getTitle(), expectedTitle);
}

```

Q104. Why can `@Test(timeOut = ...)` break a Selenium test? TestNG runs a test that has a `timeOut` on a separate thread. A driver kept in a `ThreadLocal` was started on the original thread, so the test thread finds no browser (No browser started on this thread). That is why the framework does not use `timeOut` on browser tests. It relies on explicit waits, which fail with a clear `TimeoutException`.